C for C++ Programmers

April 03, 2026

1. Introduction

In the beginning, knowing C++ automatically meant you knew C. The original C++ compiler, `cfront`, literally translated C++ code into C before compiling it. But modern C++ has diverged dramatically from C. You have `std::string`, `std::vector`, smart pointers, RAII, templates, `iostream`, and exceptions — none of which exist in C. If someone hands you a C codebase today, your modern C++ instincts will not get you very far.

So why learn C? Because C is everywhere. Operating systems, embedded firmware, database engines, language runtimes — the foundational software that the world runs on is written in C. Even if you spend most of your career in C++, you will encounter C code, C libraries, and C APIs. Understanding C makes you a better programmer, period.

My go-to C textbook is *The C Programming Language (Second Edition)* by Brian Kernighan and Dennis Ritchie — often called "K&R." It is one of the most influential programming books ever written, and it opens like this:

> ### Getting Started
>
> The only way to learn a new programming language is by writing programs in it. The first program to write is the same for all languages: Print the words hello, world. This is the big hurdle; to leap over it you have to be able to create the program text somewhere, compile it successfully, load it, run it, and find out where your output went. With these mechanical details mastered, everything else is comparatively easy.
>
> In C, the program to print "hello, world" is
>
> ```c
> #include <stdio.h>
> main()
> {
>  printf("hello, world\n");
> }
> ```
>
> Just how to run this program depends on the system you are using. As a specific example, on the UNIX operating system you must create the program in a file whose name ends in ".c", such as hello.c, then compile it with the command `cc hello.c`. If you haven't botched anything, such as omitting a character or misspelling something, the compilation will proceed silently, and make an executable file called a.out. If you run a.out by typing the command `a.out`, it will print `hello, world`. On other systems, the rules will be different; check with a local expert.

Differences Summary

Notice the differences from C++. There is no `#include <iostream>`, no `std::println`, no `std::cout`. In C, you use `printf` from `<stdio.h>` for output. The file ends in `.c`, not `.cpp`. You compile with `cc` (the C compiler) rather than `c++`.

Here is a quick summary of the biggest differences you will encounter:

C++	C
`std::string`	`char` arrays with `'\0'`
`std::vector`	raw arrays or `malloc`
`std::cout` / `std::println`	`printf`
`std::cin`	`scanf`
`new` / `delete`	`malloc` / `free`
Smart pointers	Raw pointers (only option)
Classes and objects	Structs and functions
References (`&`)	Pointers (`*`)
`bool` (built-in)	`_Bool` or `#include <stdbool.h>`
`// comments`	`/* comments */` (C89); `//` allowed since C99

Tip: C source files use the `.c` extension and are compiled with `cc`. If you accidentally compile a `.c` file with `c++`, it will be treated as C++ and may accept syntax that real C compilers reject. Always use `cc` when writing C.

Printing with `printf`

```
int printf(const char *format, ...);
```

In C++, you use `std::cout` or `std::println` for output. In C, you use `printf` from `<stdio.h>`. Unlike `std::println`, `printf` does not automatically add a newline at the end of the output. If you want each call to end on its own line, you must include `\n` in the format string yourself.

The first argument to `printf` is a **format string** containing literal text and **format specifiers** that start with `%`. Each specifier is replaced by the corresponding argument that follows:

```
int year = 1984;
printf("Year: %d\n", year);   // Year: 1984
```

Here are the format specifiers you will use most:

Specifier	Type	Example	Output
`%d`	`int` (decimal)	`printf("%d", 42)`	`42`
`%x`	`int` (hex, lowercase)	`printf("%x", 255)`	`ff`
`%X`	`int` (hex, uppercase)	`printf("%X", 255)`	`FF`
`%f`	`double`	`printf("%f", 3.14)`	`3.140000`
`%e`	`double` (scientific)	`printf("%e", 3.14)`	`3.140000e+00`
`%c`	`char`	`printf("%c", 'A')`	`A`
`%s`	`char *` (string)	`printf("%s", "hola")`	`hola`
`%p`	pointer	`printf("%p", (void *)ptr)`	`0x7ffd...`
`%zu`	`size_t`	`printf("%zu", sizeof(int))`	`4`

Trap: The first argument to `printf` should always be a string literal. Never pass a variable as the first argument. It may work, but it is a potential security vulnerability (format string attack). If you only want to print a string variable, don't do ~~`printf(str)`~~, do `printf("%s", str)`.

You can control the width and precision of output by placing numbers between the `%` and the specifier letter. A number before the specifier sets the minimum field width, and a `.` followed by a number sets the precision (decimal places for floats, max characters for strings):

```c
double score = 98.6;
printf("Score: %f\n", score);      // 98.600000  (default: 6 decimal places)
printf("Score: %.2f\n", score);    // 98.60      (2 decimal places)
printf("Score: %e\n", score);      // 9.860000e+01 (scientific notation)
```

Zero-filled output is useful for track numbers, timestamps, and hex addresses. Place a `0` before the width to pad with zeros instead of spaces:

```c
for (int i = 1; i <= 5; i++) {
    printf("Track %02d\n", i);
}
// Track 01
// Track 02
// Track 03
// Track 04
// Track 05

int color = 0xFF8800;
printf("Color: 0x%06X\n", color);   // Color: 0xFF8800

int score = 95;
printf("Score: %d%%\n", score);      // Score: 95%
```

Since % introduces a format specifier, you must write %% to print a literal percent sign.

> **Trap:** `printf` does not check that your format specifiers match the types of your arguments. If you write `printf("%d", 3.14)`, the compiler may warn you, but it will not stop you. The result is garbage. Always match specifiers to types: `%d` for `int`, `%f` for `double`, `%s` for `char *`, and so on.

Try It: printf Starter

```c
#include <stdio.h>

int main(void) {
    // Basic format specifiers
    printf("Integer: %d\n", 1984);
    printf("Hex:     %x (lowercase)  %X (uppercase)\n", 255, 255);
    printf("Float:   %f\n", 3.14);
    printf("Sci:     %e\n", 3.14);
    printf("Char:    %c\n", 'A');
    printf("String:  %s\n", "Hola");

    // Width and precision
    printf("\nFormatted output:\n");
    for (int i = 1; i <= 3; i++)
        printf("  Track %02d\n", i);

    printf("  Pi: %.2f\n", 3.14159);
    printf("  100%% complete\n");

    return 0;
}
```

Key Points

- C uses `printf` from `<stdio.h>` for output — there is no `std::cout` or `std::println`.

- Format specifiers (`%d`, `%s`, `%f`, etc.) must match the types of the arguments passed to `printf`.
- C source files end in `.c` and are compiled with `cc`, not `c++`.
- C has no classes, no templates, no exceptions, no smart pointers, and no `std::string`.
- `printf` does not add a newline automatically — you must include `\n` yourself.

Exercises

1. **Think about it:** C uses format specifiers in `printf` while C++ uses `operator<<` or `std::format`. What advantage does the format string approach give you when writing output to a log file? What is a disadvantage?

2. **What does this print?**

   ```
   printf("%05d %x\n", 42, 255);
   ```

3. **Calculation:** How many bytes does the string literal `"hello"` occupy in memory?

4. **Where is the bug?**

   ```
   double pi = 3.14159;
   printf("Pi is %d\n", pi);
   ```

5. **Write a program** that prints a 5x5 multiplication table using `printf` with width formatting so the columns are aligned.